

# Final Project

## DEEP LEARNING

**Student:** Sam COLLIN, [sam.collin@mail.concordia.ca](mailto:sam.collin@mail.concordia.ca)

**Lecturer:** Kefaya QADDOUM, [kefaya.qaddoum@concordia.ca](mailto:kefaya.qaddoum@concordia.ca)

### Project Overview:

In this project we will have two challenges. They will both involve learning from limited data. Note a big part of this assignment is based on properly reporting the approaches and development stages as well as a literature review.

### Division of work in the team:

I did this project entirely alone. All the decisions, approaches and results are mine.

### Personal Notes:

As mentioned in the overview, the approach is important. In the following project and especially in the external code, you will see that I really spent time on analyzing the results and try a massive number of configurations. I might not had the time to explore as many method as I would have wanted but I feel that i learned a lot on hparams search and ensuring visibility and tracability with my code. I used Weights and Biases to really dig into the results and figure out what would be the best solution possible given what I've tried. I hope this effort in experimentation and rigorous tracking will be valued during grading, just as much as trying out well-known methods.

### Weights & Biases:

For experiment tracking and analysis, I used Weights & Biases (W&B), a tool designed to monitor machine learning runs, compare models, and visualize metrics over time. It was especially helpful for keeping track of the many experiments I conducted during this project. Although I will go into more detail later when relevant, a full implementation using W&B is provided in the appendix code. Note that reproducing those specific runs requires a personal W&B account. However, the main submission code remains self-contained and only implements the two best methods for each challenge, without any dependency on W&B.

## 1 Challenge 1

### Overview of challenge 1

Few-sample learning on the CIFAR-10 dataset images. We will consider a dataset with only 25 samples per class and 2 object classes.

For testing, a larger dataset will be used (e.g. the CIFAR-10 test set). The job is to to make an algorithm that will train on 2 randomly selected classes from a CIFAR train set with 50 samples. We may not use any model trained on external data or any external data for our answer.

On the other hand, we may consider any kind of approach without constraint including non-deep learning methods, combinations of feature extraction, etc as long as it only utilizes the 50 examples for training.

## Code

We will first start from the baseline provided model. It is an interesting model as it is a CNN. We know that CNN are particularly well adapted for images as it captures local-dependencies. It makes it a perfect candidate to start our analysis. In case of a few-shot application as this one, we really need to ensure our model is not too large simply because we don't want to overfit. The first baseline model is named "Net\_baseline" and has the following architecture consisting of:

- A Conv2d layer with 3 input channels, 16 output channels, and a kernel size of 3, followed by a ReLU activation.
- A Conv2d layer with 16 input channels, 16 output channels, a kernel size of 3, and a stride of 2, followed by a ReLU activation.
- A Conv2d layer with 16 input channels, 32 output channels, and a kernel size of 3, followed by a ReLU activation.
- A Conv2d layer with 32 input channels, 32 output channels, a kernel size of 3, and a stride of 2, followed by a ReLU activation.
- A fully connected (Linear) layer mapping from  $32 \times 5 \times 5$  features to 10 output classes.

```
1 class Net_baseline(torch.nn.Module):
2     def __init__(self):
3         super(Net_baseline, self).__init__()
4         self.layers = nn.ModuleList()
5
6         self.layers+=[nn.Conv2d(3, 16, kernel_size=3) ,
7                        nn.ReLU(inplace=True)]
8         self.layers+=[nn.Conv2d(16, 16, kernel_size=3, stride=2),
9                        nn.ReLU(inplace=True)]
10        self.layers+=[nn.Conv2d(16, 32, kernel_size=3),
11                       nn.ReLU(inplace=True)]
12        self.layers+=[nn.Conv2d(32, 32, kernel_size=3, stride=2),
13                       nn.ReLU(inplace=True)]
14        self.fc = nn.Linear(32*5*5, 10)
15
16    def forward(self, x):
17        for i in range(len(self.layers)):
18            x = self.layers[i](x)
19            x = x.view(-1, 32*5*5)
20            x = self.fc(x)
21        return x
```

**Listing 1:** Baseline Model Architecture

The training and test functions are the following:

*(Note1: Note that the loss has been change in the training function to ensure consistency with the test and have an average loss instead of the last batch loss.)*

*(Note2: Note that I added a line "wandb.log" at the end of each function. It passes the metrics like train&test loss to the wandb framework to monitor them later through panels.)*

```
1 def train(model, device, train_loader, optimizer, epoch, display=True):
2     model.train()
3     running_loss = 0.
```

```

4     for batch_idx, (data, target) in enumerate(train_loader):
5         data, target = data.to(device), target.to(device)
6         optimizer.zero_grad()
7         output = model(data)
8         loss = F.cross_entropy(output, target)
9         loss.backward()
10        optimizer.step()
11        running_loss += loss.item()
12
13    # Slight modif to the loss to have the mean over all the batches instead of
14    # the last one
15    avg_loss = running_loss / len(train_loader)
16
17    if display:
18        print('Train Epoch: {} [{} / {}] ({:.0f}%) \t Loss: {:.6f}'.format(
19            epoch, batch_idx * len(data), len(train_loader.dataset),
20            100. * batch_idx / len(train_loader), avg_loss))
21    wandb.log({"train_loss": avg_loss, "epoch": epoch})
22
23 def test(model, device, test_loader):
24     model.eval()
25     test_loss = 0
26     correct = 0
27     with torch.no_grad():
28         for data, target in test_loader:
29             data, target = data.to(device), target.to(device)
30             output = model(data)
31             test_loss += F.cross_entropy(output, target, size_average=False).item()
32             # sum up batch loss
33             pred = output.max(1, keepdim=True)[1] # get the index of the max log-
34             # probability
35             correct += pred.eq(target.view_as(pred)).sum().item()
36
37     test_loss /= len(test_loader.dataset)
38     accuracy = 100. * correct / len(test_loader.dataset)
39     print('\nTest set: Average loss: {:.4f}, Accuracy: {} / {} ({:.2f}%) \n'.format(
40         test_loss, correct, len(test_loader.dataset),
41         accuracy))
42     wandb.log({"test_loss": test_loss, "test_accuracy": accuracy})
43     return accuracy

```

In the following cell, data is imported and transformed.

Regarding the validation set, only normalization is applied to ensure compatibility with training.

Regarding the training set 5 augmentation are proposed.

We'll later analyze the results and see if one is better than another.

(Note: Here “transform\_train\_0” means no data augmentation)

```

1 from numpy.random import RandomState
2 import numpy as np
3 import torch.optim as optim
4 from torch.utils.data import Subset
5
6 from torchvision import datasets, transforms
7 normalize = transforms.Normalize((0.4914, 0.4822, 0.4465), (0.247, 0.243, 0.261))

```

```

8
9 use_cuda = torch.cuda.is_available()
10 device = torch.device("cuda" if use_cuda else "cpu")
11
12 transform_val = transforms.Compose([transforms.ToTensor(), normalize]) #careful to
    keep this one same
13 #We need two copies of this due to weird dataset api
14 cifar_data_val = datasets.CIFAR10(root='.',train=True, transform=transform_val,
    download=True)
15
16
17 ## Data augmentation ##
18
19 # Baseline (no augmentation)
20 transform_train_0 = transforms.Compose([transforms.ToTensor(), normalize])
21 cifar_data_0 = datasets.CIFAR10(root='.',train=True, transform=transform_train_0,
    download=True)
22
23 # Cropping + Flip
24 transform_train_1 = transforms.Compose([
25     transforms.RandomCrop(32, padding=4),      # random cropping with fill
26     transforms.RandomHorizontalFlip(),         # rdm horizontal flip
27     transforms.ToTensor(),
28     normalize
29 ])
30
31 # Rotation + Color
32 transform_train_2 = transforms.Compose([
33     transforms.RandomRotation(15), # Random rotation in the range [-15, 15]
34     degrees
35     transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
36     transforms.ToTensor(),
37     normalize
38 ])
39
40 # Random Affine
41 transform_train_3 = transforms.Compose([
42     transforms.RandomAffine(
43         degrees=10,          # Rotation in a narrower range
44         translate=(0.1, 0.1), # Horizontal and vertical translation (10% of the
45                               image)
46         scale=(0.9, 1.1),    # Scale varying between 90% and 110%
47         shear=10             # Maximum shear of 10 degrees
48     ),
49     transforms.ToTensor(),
50     normalize
51 ])
52
53 transform_train_4 = transforms.Compose([
54     transforms.RandomHorizontalFlip(), # You can keep a flip as a precaution
55     transforms.ToTensor(),
56     normalize,
57     transforms.RandomErasing(
58         p=0.5,                # Probability of deletion
59         scale=(0.02, 0.33)    # Size of deleted area as a proportion of the
60                               image
61     )
62 ])

```

```

61 transform_train_5 = transforms.Compose([
62     transforms.RandomHorizontalFlip(),
63     transforms.GaussianBlur(kernel_size=3), # Applying Gaussian blur with a 3x3
        kernel
64     transforms.ToTensor(),
65     normalize
66 ])
67
68
69 transform_dict = {
70     0: transform_train_0,
71     1: transform_train_1,
72     2: transform_train_2,
73     3: transform_train_3,
74     4: transform_train_4,
75     5: transform_train_5
76 }

```

Finally, this is our main function. Before running it, we have to make sure that the runs are well configured.

The goal here is to train a certain configuration for a given number of seeds to average the results as the variance might be very high for few-shots applications.

The configuration instantiated by “wandb.init()“ allows to define all the hyperparameters that we want for our model but also any specific variable that could be needed to describe or differentiate a run from another. We can give a name, and associate with a certain group or tags. By entering this information correctly, we can easily manage the results in the graphical interface and group them as we want to visualize differences.

The steps are the following:

- Loop over each seed
- Define the configuration (The same will remain for all the seed except the name of the run)
- Apply data augmentation if needed and pick random samples to create the subsets of 25 samples per class for training and 200 for test.
- Instantiate the model and the optimizer
- Train and test the actual configuration
- Display the final mean accuracy over all seeds and compute the std

```

1 accs = []
2 n_seeds = 12
3
4 for seed in range(1, n_seeds+1):
5
6     # Separate the runs in wandb to compare easily:
7     wandb.init(
8         project="Challenge1_DL_v6",
9         name=f"Run_seed_{seed}",
10        group="Baseline_OPTIMIZED_REGULARIZED",
11        tags=["Baseline_OPTIMIZED_REGULARIZED"],

```

```

12     config={
13         "seed": seed,
14         "epochs": 100,
15         "batch_size": 32,
16         "lr": 0.01,
17         "momentum": 0.8,
18         "weight_decay": 0.001,
19         "model": "Net_complex",
20         "dropout_prob": 0.2,
21         "data_augmentation": 1    # zero means no augmentation
22     },
23     reinit=True
24 )
25
26 prng = RandomState(seed)
27 random_permute = prng.permutation(np.arange(0, 1000))
28 classes = prng.permutation(np.arange(0,10))
29
30 cifar_data = datasets.CIFAR10(
31     root='.',
32     train=True,
33     transform=transform_dict[wandb.config.data_augmentation],
34     download=True
35 )
36
37 indx_train = np.concatenate([np.where(np.array(cifar_data.targets) == classe)
38     [0][random_permute[0:25]] for classe in classes[0:2]])
39
40 indx_val = np.concatenate([np.where(np.array(cifar_data.targets) == classe)[0][
41     random_permute[25:225]] for classe in classes[0:2]])
42
43 train_data = Subset(cifar_data, indx_train)
44 val_data = Subset(cifar_data_val, indx_val)
45
46 print('Num_Samples_For_Training%d Num_Samples_For_Val%d'%(train_data.indices.
47     shape[0], val_data.indices.shape[0]))
48
49 train_loader = torch.utils.data.DataLoader(train_data,
50     batch_size=wandb.config.batch_size,
51     shuffle=True)
52
53 val_loader = torch.utils.data.DataLoader(val_data,
54     batch_size=wandb.config.batch_size,
55     shuffle=False)
56
57 model = Net_wide_BatchNorm_Dropout(dropout_prob=wandb.config.dropout_prob)
58 model.to(device)
59 optimizer = torch.optim.SGD(model.parameters(), lr=wandb.config.lr, momentum=
60     wandb.config.momentum,
61     weight_decay=wandb.config.weight_decay)
62
63 for epoch in range(wandb.config.epochs):
64     train(model, device, train_loader, optimizer, epoch, display=epoch%5==0)
65
66 accs.append(test(model, device, val_loader))
67 wandb.finish()
68
69 accs = np.array(accs)
70 print(f'Acc_over_{n_seeds}_instances: %.2f+-%.2f'%(accs.mean(), accs.std()))

```

---

## Listing 2: Optimal configuration for Challenge 1

The code above is the best configuration i reached for Challenge 1. Below, I will explain step by step how I reached this one by optimizing both the architectures and the hyperparameters and show the results.

### Results and Optimization

#### 1.0.1 Baseline

Thanks to wandb.ai, we can now visualize our results. The first step is to run our **baseline** for twelve different seeds.

To recall, our baseline is:

- "epochs": 100
- "batch\_size": 128
- "lr": 0.01
- "momentum": 0.9
- "weight\_decay": 0.0005
- "model": "Net\_baseline"
- "data\_augmentation": 0 # zero means no augmentation

The following results are obtained:



Here, one can see each seed of the configuration runned. For the rest of the study and to enforce the analysis we will work with the mean of these 12 seeds per config. So, once grouped the baseline configuration is represented by the following metrics:



The baseline has a final training loss of 0.23 and a test loss of 1.5

The averaged test accuracy for our baseline is **62.38** with a standard deviation  $\sigma$  of **8.6**

## Hyperparameter tuning

To see if our model can improve, before using heavy techniques we can study the role of the hparams and see what's their role on the model.

### Learning rate

Let's start by the learning rate used to optimize with SGD. Different values have been tested such as 0.001, 0.005, 0.01 (baseline), and 0.05. Their result comparison is the following:



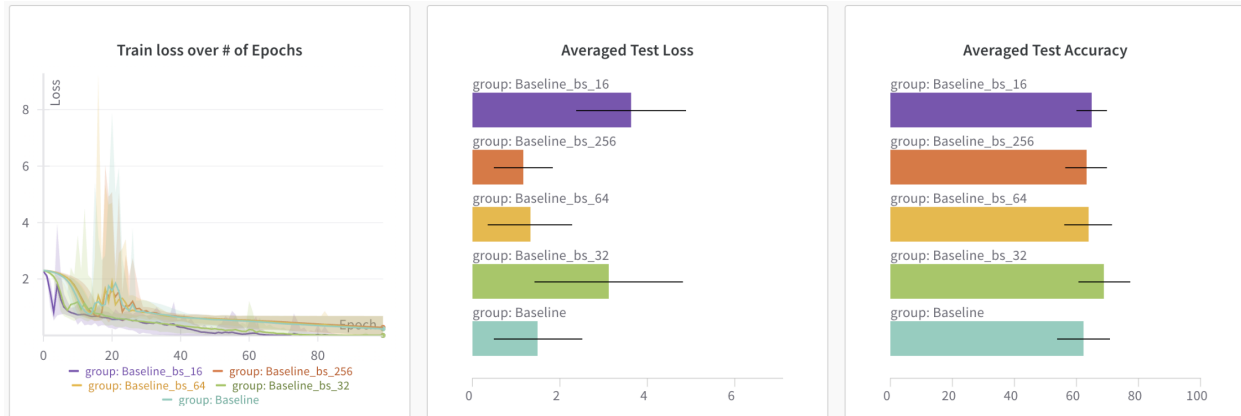
One can see that, despite 0.05, the result are almost the same in terms of accuracy. To continue simply, **we'll stick with lr=0.01** as in the baseline.



## Batch\_size

Let's try different batch size to see what's best:

Tested values are 16, 32, 64, 128 (baseline), 256:



These results may seem counter-intuitive. Indeed, the best accuracy seems to have a higher loss. A larger batch size provides a more stable estimator of the gradient, which can result in more consistent convergence in terms of loss, but sometimes less generalisation, or vice versa. A smaller batch size introduces more noise into the updating of the weights (stochasticity), which can encourage better exploration of the parameter space.

It might need further exploration but for now, **we'll retain 32 for the optimal model**. Small batch size for few shot is often a good idea.

## Momentum

The tested values for the momentum are : 0,7,0.8, 0.85, 0.9 (baseline), 0.95:



At first sight, momentum 0.95 has the best results. However when we look closer at the test loss and the accuracy's standard deviation, one can see that 0.8 might look more consistent and promising. **We will use 0.8 for the optimized model.**

## Weight decay

The tested values are : 0.0001, 0.0005 (baseline), 0.001



The results are similar, regarding the std, it seems that **0.001** looks a bit more consistent.

### 1.0.2 Architecture

It is now time to vary the architecture.

The first variation Baseline\_deep is a deeper network. From the original model, we added 1 layer :

- A Conv2d layer with 16 input channels, 16 output channels, a kernel size of 3, and a stride of 2, followed by a ReLU activation.

The second variation Baseline\_wide is a wider network. From the original model, we added more neurons :

- The number of channels has been multiplied by two

```
1 # Deeper Model
2 class Net_deep(torch.nn.Module):
3     def __init__(self):
4         super(Net_deep, self).__init__()
5         self.layers = nn.ModuleList()
6
7         self.layers += [nn.Conv2d(3, 16, kernel_size=3), nn.ReLU(inplace=True)]
8         self.layers += [nn.Conv2d(16, 16, kernel_size=3, stride=2), nn.ReLU(
9             inplace=True)]
10        # Adding an extra layer
11        self.layers += [nn.Conv2d(16, 16, kernel_size=3), nn.ReLU(inplace=True)]
12        self.layers += [nn.Conv2d(16, 32, kernel_size=3), nn.ReLU(inplace=True)]
13        self.layers += [nn.Conv2d(32, 32, kernel_size=3, stride=2), nn.ReLU(
14            inplace=True)]
15        self.fc = nn.Linear(32*4*4, 10)
16
17    def forward(self, x):
18        for layer in self.layers:
19            x = layer(x)
20        x = x.view(x.size(0), -1)
```

```

19         x = self.fc(x)
20         return x
21
22 # Wider Model
23 class Net_wide(torch.nn.Module):
24     def __init__(self):
25         super(Net_wide, self).__init__()
26         self.layers = nn.ModuleList()
27
28         self.layers += [nn.Conv2d(3, 32, kernel_size=3), nn.ReLU(inplace=True)]
29         self.layers += [nn.Conv2d(32, 32, kernel_size=3, stride=2), nn.ReLU(
30             inplace=True)]
31         self.layers += [nn.Conv2d(32, 64, kernel_size=3), nn.ReLU(inplace=True)]
32         self.layers += [nn.Conv2d(64, 64, kernel_size=3, stride=2), nn.ReLU(
33             inplace=True)]
34         self.fc = nn.Linear(64*5*5, 10)
35
36     def forward(self, x):
37         for layer in self.layers:
38             x = layer(x)
39         x = x.view(x.size(0), -1)
40         x = self.fc(x)
41         return x

```

Listing 3: Wide and Deep Architecture



In a few-shot scenario with very limited data, deeper architecture tends to increase the risk of overfitting and training instability due to its additional layers and parameters, it makes it more likely to memorize the few available examples rather than learn robust features.

On the other hand, widening the network—by increasing the number of channels in each layer—enhances its representational capacity without substantially increasing its depth, enabling it to capture richer features and generalize better while still maintaining a relatively stable training process.

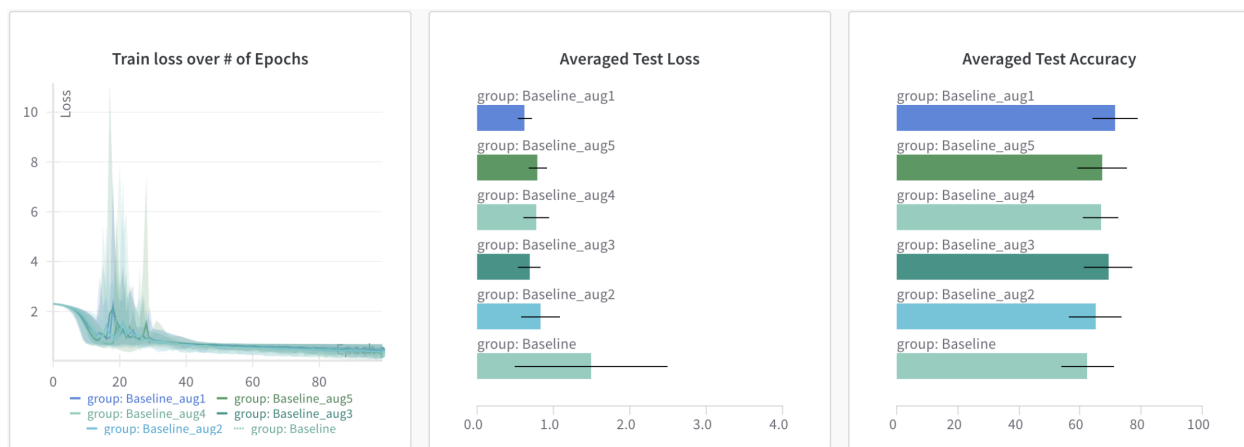
For now, **the wide architecture will be retained.**

### 1.0.3 Data augmentation

Finally, we introduce data augmentation. In a few-shot context, it can really make a huge difference. Data is modified to be a bit noisier and the model should be more robust in the end. The techniques are the following :

- **Data augmentation 0 (baseline)** : uses the original images with only basic normalization as a baseline.
- **Data augmentation 1** : applies random cropping with padding and random horizontal flipping, introducing variations in object position and mirror invariance.
- **Data augmentation 2** : combines random rotation (up to  $15^\circ$ ) with color jitter (adjusting brightness, contrast, saturation, and hue) to simulate changes in viewpoint and lighting conditions.
- **Data augmentation 3** : employs random affine transformations—including slight rotations, translations, scaling, and shearing—to mimic geometric distortions.
- **Data augmentation 4** : involves a random horizontal flip followed by random erasing, which occludes a random portion of the image to simulate noise or partial occlusion.
- **Data augmentation 5** : applies a random horizontal flip coupled with Gaussian blur (using a  $3 \times 3$  kernel) to emulate slight out-of-focus effects.

All these transformations parameters are chosen arbitrarily. The goal is to see the effect of each perturbation and choose the most promising.



One can see that any techniques, whatever it is, outperforms the baseline. It emphasizes the importance of data augmentation and the benefits it brings.

Regarding the results, **augmentation 1 seems to be the most promising one**. This is the one we choose.

### 1.0.4 Optimized Baseline

From now on, all the parameters were evaluated individually. It is time to combine them to see if we outperform the baseline. The final optimized configurations is :

- "epochs": 100
- "batch\_size": 32
- "lr": 0.01
- "momentum": 0.8
- "weight\_decay": 0.001
- "model": "Baseline\_wide"
- "data\_augmentation": 1 # zero means no augmentation



The results are way higher !

**The test loss is of 0.7 for the optimized against 1.5 for the baseline. For the test accuracy, The optimized versions reaches a mean of 74.6% and a std of 8.3 against 62.4 with a std of 8.6 for the baseline.**

It is around twelve points higher. These improvements suggests that the diversified augmentation techniques and hparams tuning allow the model to learn more robust features, which in turn improves its generalization ability in a few-shot scenario while slightly reducing the variability in performance.

To ensure correct results, the optimized configuration has been run on 50 seeds.

### 1.0.5 Regularization

In our few-shot scenario, regularization might be very efficient or useful.

We tried different techniques and combination below such as adding BatchNorm, GroupNorm and Dropout.

```
1  # Wider Network with Batch Normalization
2  class Net_wide_BatchNorm(nn.Module):
3      def __init__(self):
4          super(Net_wide_BatchNorm, self).__init__()
5          self.conv1 = nn.Conv2d(3, 32, kernel_size=3)
6          self.bn1 = nn.BatchNorm2d(32)
7          self.conv2 = nn.Conv2d(32, 32, kernel_size=3, stride=2)
8          self.bn2 = nn.BatchNorm2d(32)
9          self.conv3 = nn.Conv2d(32, 64, kernel_size=3)
10         self.bn3 = nn.BatchNorm2d(64)
11         self.conv4 = nn.Conv2d(64, 64, kernel_size=3, stride=2)
12         self.bn4 = nn.BatchNorm2d(64)
13         self.fc = nn.Linear(64*5*5, 10)
14
15     def forward(self, x):
16         x = F.relu(self.bn1(self.conv1(x)))
17         x = F.relu(self.bn2(self.conv2(x)))
18         x = F.relu(self.bn3(self.conv3(x)))
19         x = F.relu(self.bn4(self.conv4(x)))
20         x = x.view(x.size(0), -1)
21         x = self.fc(x)
22         return x
23
24  # Wider Network with Dropout
25  class Net_wide_Dropout(nn.Module):
26      def __init__(self, dropout_prob=0.5):
27          super(Net_wide_Dropout, self).__init__()
28          self.conv1 = nn.Conv2d(3, 32, kernel_size=3)
29          self.conv2 = nn.Conv2d(32, 32, kernel_size=3, stride=2)
30          self.conv3 = nn.Conv2d(32, 64, kernel_size=3)
31          self.conv4 = nn.Conv2d(64, 64, kernel_size=3, stride=2)
32          self.fc = nn.Linear(64*5*5, 10)
33          self.dropout = nn.Dropout(dropout_prob)
34
35     def forward(self, x):
36         x = F.relu(self.conv1(x))
37         x = self.dropout(x)
38         x = F.relu(self.conv2(x))
39         x = self.dropout(x)
40         x = F.relu(self.conv3(x))
41         x = self.dropout(x)
42         x = F.relu(self.conv4(x))
43         x = self.dropout(x)
44         x = x.view(x.size(0), -1)
45         x = self.fc(x)
46         return x
47
48  # Wider Network with Batch Normalization and Dropout
49  class Net_wide_BatchNorm_Dropout(nn.Module):
50      def __init__(self, dropout_prob=0.5):
51          super(Net_wide_BatchNorm_Dropout, self).__init__()
```

```

52     self.conv1 = nn.Conv2d(3, 32, kernel_size=3)
53     self.bn1 = nn.BatchNorm2d(32)
54     self.conv2 = nn.Conv2d(32, 32, kernel_size=3, stride=2)
55     self.bn2 = nn.BatchNorm2d(32)
56     self.conv3 = nn.Conv2d(32, 64, kernel_size=3)
57     self.bn3 = nn.BatchNorm2d(64)
58     self.conv4 = nn.Conv2d(64, 64, kernel_size=3, stride=2)
59     self.bn4 = nn.BatchNorm2d(64)
60     self.fc = nn.Linear(64*5*5, 10)
61     self.dropout = nn.Dropout(dropout_prob)
62
63     def forward(self, x):
64         x = F.relu(self.bn1(self.conv1(x)))
65         x = self.dropout(x)
66         x = F.relu(self.bn2(self.conv2(x)))
67         x = self.dropout(x)
68         x = F.relu(self.bn3(self.conv3(x)))
69         x = self.dropout(x)
70         x = F.relu(self.bn4(self.conv4(x)))
71         x = self.dropout(x)
72         x = x.view(x.size(0), -1)
73         x = self.fc(x)
74         return x
75
76 # Wider Network with Group Normalization
77 class Net_wide_GroupNorm(nn.Module):
78     def __init__(self, num_groups=8, dropout_prob=0.5):
79         super(Net_wide_GroupNorm, self).__init__()
80
81         self.conv1 = nn.Conv2d(3, 32, kernel_size=3)
82         self.gn1 = nn.GroupNorm(num_groups=num_groups, num_channels=32)
83
84         self.conv2 = nn.Conv2d(32, 32, kernel_size=3, stride=2)
85         self.gn2 = nn.GroupNorm(num_groups=num_groups, num_channels=32)
86
87         self.conv3 = nn.Conv2d(32, 64, kernel_size=3)
88         self.gn3 = nn.GroupNorm(num_groups=num_groups, num_channels=64)
89
90         self.conv4 = nn.Conv2d(64, 64, kernel_size=3, stride=2)
91         self.gn4 = nn.GroupNorm(num_groups=num_groups, num_channels=64)
92
93         self.fc = nn.Linear(64 * 5 * 5, 10)
94
95     def forward(self, x):
96         x = F.relu(self.gn1(self.conv1(x)))
97         x = F.relu(self.gn2(self.conv2(x)))
98         x = F.relu(self.gn3(self.conv3(x)))
99         x = F.relu(self.gn4(self.conv4(x)))
100        x = x.view(x.size(0), -1)
101        x = self.fc(x)
102        return x
103
104 # Wider Network with Group Normalization
105 class Net_wide_GroupNorm_Dropout(nn.Module):
106     def __init__(self, num_groups=8, dropout_prob=0.5):
107         super(Net_wide_GroupNorm_Dropout, self).__init__()
108
109         self.conv1 = nn.Conv2d(3, 32, kernel_size=3)
110         self.gn1 = nn.GroupNorm(num_groups=num_groups, num_channels=32)

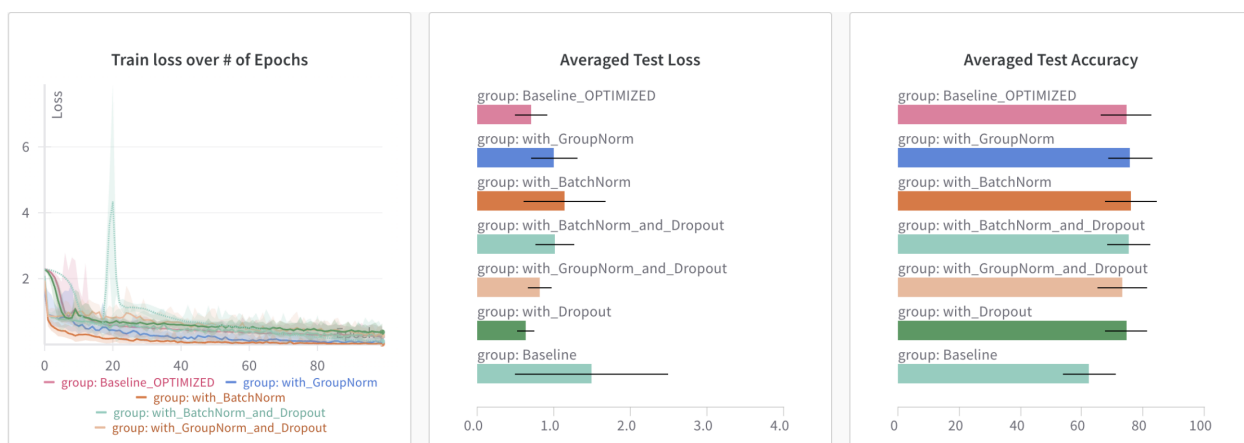
```

```

111
112     self.conv2 = nn.Conv2d(32, 32, kernel_size=3, stride=2)
113     self.gn2 = nn.GroupNorm(num_groups=num_groups, num_channels=32)
114
115     self.conv3 = nn.Conv2d(32, 64, kernel_size=3)
116     self.gn3 = nn.GroupNorm(num_groups=num_groups, num_channels=64)
117
118     self.conv4 = nn.Conv2d(64, 64, kernel_size=3, stride=2)
119     self.gn4 = nn.GroupNorm(num_groups=num_groups, num_channels=64)
120
121     self.fc = nn.Linear(64 * 5 * 5, 10)
122     self.dropout = nn.Dropout(dropout_prob)
123
124     def forward(self, x):
125         x = F.relu(self.gn1(self.conv1(x)))
126         x = self.dropout(x)
127         x = F.relu(self.gn2(self.conv2(x)))
128         x = self.dropout(x)
129         x = F.relu(self.gn3(self.conv3(x)))
130         x = self.dropout(x)
131         x = F.relu(self.gn4(self.conv4(x)))
132         x = self.dropout(x)
133         x = x.view(x.size(0), -1)
134         x = self.fc(x)
135         return x

```

Listing 4: Regularized Architectures



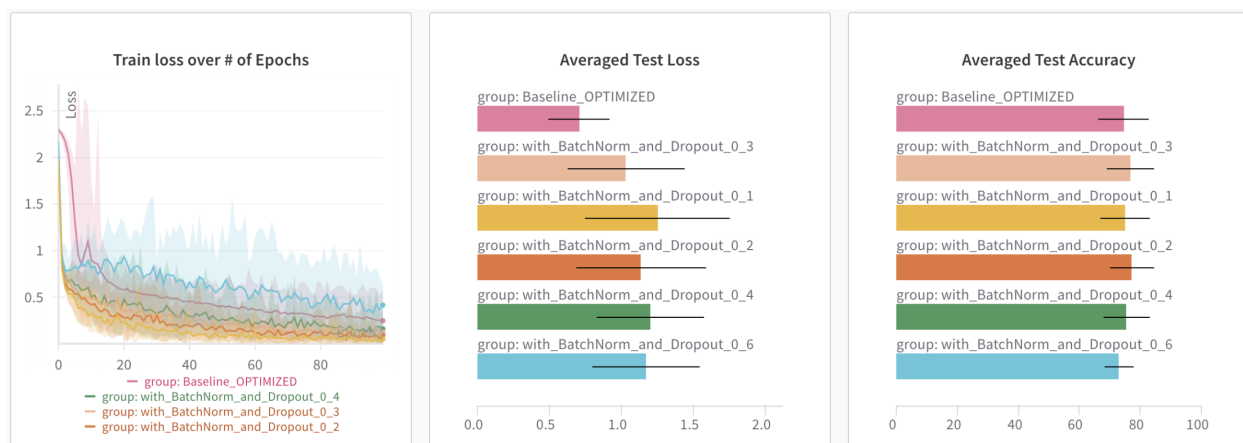
The results demonstrate that adding regularization can help the model to generalize. BatchNorm and GroupNorm alone yields better results, however, the loss on the train set is very close to 0. When using dropout, this loss remain decent. To make sure we don't overfit, **we'll use the BatchNorm with Dropout.**



## Dropout Probability

We now need to find the optimal probability of drop for the dropout layers.

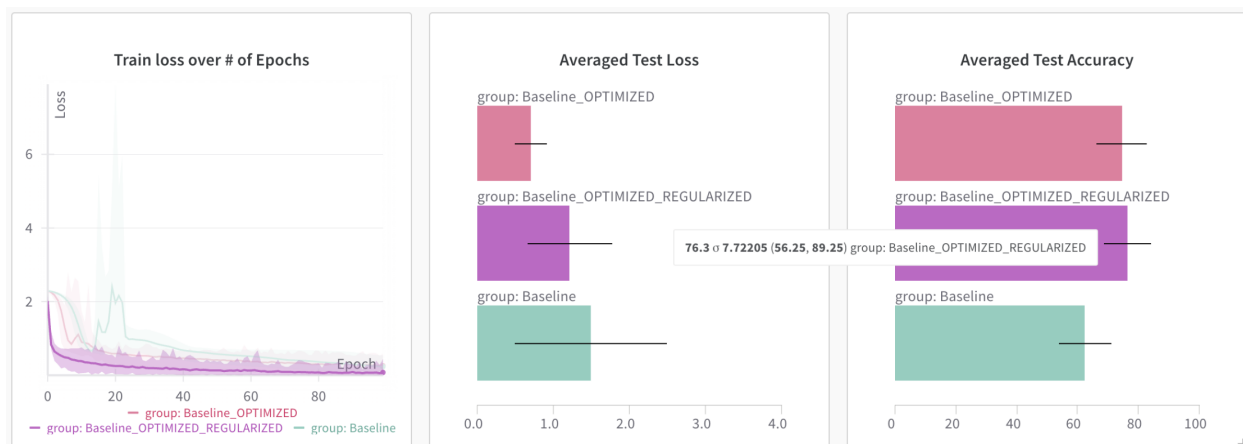
The values tested are : 0.1, 0.2, 0.3, 0.4, 0.6:



The best probability  $p$  seems to be 0.2.

### 1.0.6 Optimized & Regularized vs baseline

Combining this information, we run the model on 50 different seeds to confidently assess performance and see if there are any improvements.



Again, the results have improved! **The new mean accuracy is now 76.3% (std: 7.7).**

*Additional Note:* Although better results have been obtained in the main code, the scores reported here are kept for consistency and comparison purposes due to high variance across seeds. The current best result, as reported in the submitted main code, is:

Acc over 50 instances: 77.49 +- 7.56

## 2 Challenge 2

### Overview of challenge 2

Consider the same exercise but now with the ability to use some external data or models not trained on this same data. We may consider these external data to do for example various forms of finetuning, meta-learning, semi-supervised learning with an external dataset and other related ideas.

### Strategy

For Challenge 2, I decided to follow the fine-tuning approach, it seemed to me like a practical and effective starting point given the use of pre-trained models. I began by testing several ResNet architectures available in PyTorch, such as ResNet-18 and ResNet-50, and focused on finding the most suitable configuration for the few-shot task. I adjusted key hyperparameters like the learning rate, weight decay, and training schedule, using validation performance and W&B as explained in challenge 1.

After selecting the best setup, I also explored a more advanced method: MAML (Model-Agnostic Meta-Learning). While I was able to implement it successfully (I think), the results were not conclusive, likely due to the sensitivity of MAML to hyperparameters and the complexity of its training process.

I didn't have enough time to fully tune this method, I decided not to include it as part of the final model. However, the full implementation is available in the appendix for future work.

## Literature Review: Model-Agnostic Meta-Learning (MAML)

The MAML algorithm was introduced by Chelsea Finn, Pieter Abbeel, and Sergey Levine in their 2017 paper “*Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks*”. This method is designed for few-shot learning—where a model needs to adapt quickly to new tasks using only a few training examples. It is called “model-agnostic” because it does not depend on a specific architecture: it works with any model that can be trained with gradient descent, such as fully connected networks, CNNs, or even reinforcement learning agents.

The idea is simple but powerful: instead of training a model to solve one task very well, MAML trains it to be able to learn new tasks quickly. To do this, the model learns an initial set of parameters that are “easy to fine-tune.” In practice, this means that with just one or a few gradient steps on a small dataset (called the support set), the model should already perform well on unseen examples (the query set) from the same task.

In MAML, training is done in two loops: the inner loop adapts the model to a sampled task using a few gradient steps, and the outer loop updates the original model based on how well it performs after this quick adaptation. This is exactly what I reproduced in my own implementation: I clone the model for each task, adapt it using support data, then evaluate and update the meta-model based on the query loss.

One of the key strengths of MAML is that it separates task-specific learning from meta-learning. The inner loop performs standard supervised learning on a small batch of data (the support set), while the outer loop adjusts the initial parameters so that this learning becomes more efficient. This structure makes MAML very flexible and compatible with many domains, including regression, classification, and reinforcement learning. Another important detail is that MAML optimizes for the performance after adaptation—not before. This means it does not try to directly solve the task during meta-training, but rather to prepare the model to learn it quickly later. The algorithm also supports using multiple inner loop steps, and the paper shows that it continues to improve even beyond one gradient step, which makes it more effective than simple fine-tuning from a pre-trained model.

Compared to earlier methods like Matching Networks or LSTMs used for meta-learning (e.g., Ravi and Larochelle, 2017), MAML has two main advantages. First, it does not introduce any new model components or learned optimizers—just standard gradient descent. Second, it can be applied to many types of tasks, not only classification, which makes it a good fit for a general research project like mine.

In my work, I focused on few-shot image classification using CIFAR-10. My implementation follows closely the supervised classification setup from the MAML paper: I train the model on many 2-way classification tasks and use cross-entropy loss. I tried both ResNet-18 and ResNet-50 as backbones and ran sweeps over different learning rates and training steps.

Overall, even though MAML did not outperform other approaches in my case, it remains a solid foundation for meta-learning, and my implementation shows how it can be adapted to new datasets and architectures.

## Code

The following code implements a pipeline to for fine-tuning ResNet models on the CIFAR-10 dataset under the few-shot configuration.

### Device Selection

The first block checks for available hardware to determine whether to run the model on Apple's MPS, NVIDIA CUDA, or CPU.

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 from torch.utils.data import Subset, DataLoader
6 from torchvision import datasets, transforms, models
7 import numpy as np
8 import wandb
9 from numpy.random import RandomState
10
11 # Choose device
12 if torch.backends.mps.is_available():
13     print("Device used is MPS - Apple")
14     device = torch.device("mps")
15
16 elif torch.cuda.is_available():
17     print("Device used is CUDA - Nvidia")
18     device = torch.device("cuda")
19
20 else:
21     print("Device used is CPU")
22     device = torch.device("cpu")
```

### Dataset Preparation

The function `build_dataset` prepares the data for training and validation. It uses the CIFAR-10 dataset with custom transformations. Depending on the data augmentation setting, it either applies basic normalization or stronger augmentations like cropping and flipping. The function randomly selects 2 classes and samples 25 training images and 200 validation images per class, using a fixed random seed to ensure reproducibility.

```
1 # Build train and validation loaders for few-shot CIFAR-10
2 def build_dataset(batch_size, data_augmentation, seed):
3     normalize = transforms.Normalize((0.4914, 0.4822, 0.4465),
4                                     (0.247, 0.243, 0.261))
5     transform_val = transforms.Compose([transforms.ToTensor(), normalize])
6     transform_train = transforms.Compose([
7         transforms.RandomCrop(32, padding=4),
8         transforms.RandomHorizontalFlip(),
9         transforms.ToTensor(),
10        normalize
11    ])
12     transform_dict = {
13         0: transforms.Compose([transforms.ToTensor(), normalize]),
14         1: transform_train
15     }
```

```

16     cifar_train_full = datasets.CIFAR10(root='.', train=True,
17                                         transform=transform_dict[data_augmentation
18                                         ],
19                                         download=True)
20     cifar_val_full   = datasets.CIFAR10(root='.', train=True,
21                                         transform=transform_val,
22                                         download=True)
23
24     prng = RandomState(seed)
25     perm = prng.permutation(1000)
26     classes = prng.permutation(10)
27     # select 2 random classes
28     train_idx = np.concatenate([
29         np.where(np.array(cifar_train_full.targets) == c)[0][perm[:25]]
30         for c in classes[:2]
31     ])
32     val_idx = np.concatenate([
33         np.where(np.array(cifar_val_full.targets) == c)[0][perm[25:225]]
34         for c in classes[:2]
35     ])
36     train_loader = DataLoader(Subset(cifar_train_full, train_idx),
37                                 batch_size=batch_size, shuffle=True)
38     val_loader   = DataLoader(Subset(cifar_val_full,   val_idx),
39                                 batch_size=batch_size, shuffle=False)
40     return train_loader, val_loader

```

## Model Construction

The `build_network` function constructs a ResNet-based model for transfer learning. It supports multiple architectures (ResNet-18, 34, 50, 101) and loads them with pre-trained weights from ImageNet. It allows freezing a given number of blocks to retain general features while fine-tuning deeper layers. *The final classification layer is always replaced and trained from scratch for 10 CIFAR-10 classes.*

```

1  # Build ResNet18 with dynamic freezing
2  def build_network(model_name, freeze_layers):
3      # 1) Choix du backbone
4      if model_name == "resnet18":
5          model = models.resnet18(pretrained=True)
6      elif model_name == "resnet34":
7          model = models.resnet34(pretrained=True)
8      elif model_name == "resnet50":
9          model = models.resnet50(pretrained=True)
10     elif model_name == "resnet101":
11         model = models.resnet101(pretrained=True)
12     else:
13         raise ValueError(f"Not known model {model_name}")
14
15     # always freeze conv1 & bn1
16     for p in model.conv1.parameters(): p.requires_grad = False
17     for p in model.bn1.parameters():   p.requires_grad = False
18     # freeze first N residual blocks
19     blocks = [model.layer1, model.layer2, model.layer3, model.layer4]
20     for idx, block in enumerate(blocks, start=1):
21         freeze = idx <= freeze_layers
22         for p in block.parameters(): p.requires_grad = not freeze
23     # always unfreeze classifier
24     model.fc = nn.Linear(model.fc.in_features, 10)

```

```

25     for p in model.fc.parameters(): p.requires_grad = True
26     return model.to(device)

```

## Optimizer Definition

The function `build_optimizer` creates an SGD optimizer that only updates parameters marked as trainable. It takes standard hyperparameters such as learning rate, momentum, and weight decay.

```

1  # Build SGD optimizer
2  def build_optimizer(model, lr, momentum, weight_decay):
3      params = filter(lambda p: p.requires_grad, model.parameters())
4      return optim.SGD(params, lr=lr, momentum=momentum, weight_decay=weight_decay)

```

## Training Loop

The `train_epoch` function runs one full training epoch on the training set. It computes the cross-entropy loss, performs backpropagation, and updates the model weights. Each batch loss is logged to Weights & Biases to allow monitoring and comparison across runs.

```

1  # Single epoch training
2  def train_epoch(model, device, loader, optimizer):
3      model.train()
4      total_loss = 0.0
5      for data, target in loader:
6          data, target = data.to(device), target.to(device)
7          optimizer.zero_grad()
8          output = model(data)
9          loss = F.cross_entropy(output, target)
10         loss.backward()
11         optimizer.step()
12         total_loss += loss.item()
13         wandb.log({"batch_loss": loss.item()})
14     return total_loss / len(loader)

```

## Evaluation

The `evaluate` function computes the loss and classification accuracy on the validation set.

```

1  # Evaluation on validation set
2  def evaluate(model, device, loader):
3      model.eval()
4      test_loss = 0.0
5      correct = 0
6      with torch.no_grad():
7          for data, target in loader:
8              data, target = data.to(device), target.to(device)
9              output = model(data)
10             test_loss += F.cross_entropy(output, target, reduction='sum').item()
11             pred = output.argmax(dim=1, keepdim=True)
12             correct += pred.eq(target.view_as(pred)).sum().item()
13     test_loss /= len(loader.dataset)
14     accuracy = 100. * correct / len(loader.dataset)
15     return test_loss, accuracy

```

This modular structure allows flexible experimentation with various models, hyperparameters, and training strategies. While the main submission includes only the best models, this full version particularly when used with Weights & Biases—enables deeper analysis and fine-tuning.

## Results and Optimization

To gather results here, I performed a W&B sweep, which allowed me to automatically explore multiple combinations of hyperparameters. A sweep is a process that runs many training jobs with different configurations in parallel or sequentially, helping identify the best-performing setups. It is equivalent to performing a grid-random-bayes search. The sweep configuration I used is shown below:

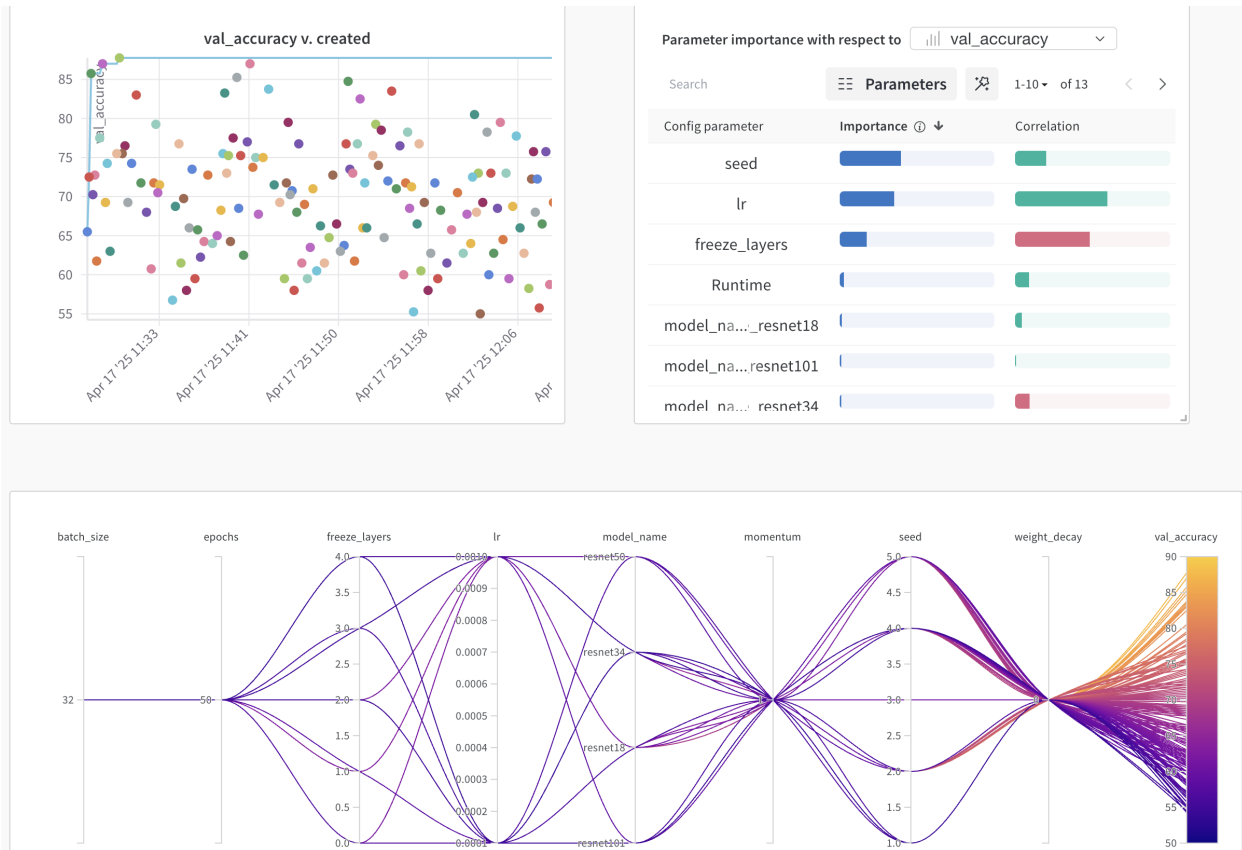
```
1 import pprint
2
3 # GRID SEARCH PRETRAINED
4
5 # Search method
6 sweep_config = {
7     'method': 'grid',
8     'name': "PreTrained_finetune_sweep"
9 }
10
11 # Objective
12 metric = {
13     'name': 'val_accuracy',
14     'goal': 'maximize'
15 }
16 sweep_config['metric'] = metric
17
18 # Parameters
19 parameters_dict = {
20     'seed': {
21         'values': [1,2,3,4,5]
22     },
23     'lr': {
24         'values': [0.001, 0.0001]
25     },
26     'batch_size': {
27         'values': [32]
28     },
29     'model_name': {
30         'values': ["resnet18", "resnet34", "resnet50", "resnet101"]
31     },
32     'freeze_layers': {
33         'values': [0, 1, 2, 3, 4]
34     },
35     'momentum': {
36         'values': [0.9]
37     },
38     'weight_decay': {
39         'values': [0.0005]
40     },
41     'epochs': {
42         'values': [50]
43     }
44 }
```

```
44 }
45 sweep_config['parameters'] = parameters_dict
46
47 parameters_dict.update({
48     'data_augmentation': {
49         'value': 1}
50 })
51
52 pprint.pprint(sweep_config)
```

As we can see, there are some parameters that are fixed and others that might change. The method here is a grid search, this will cover all the configurations (200).



The following graphs are obtained:



The bottom plot shows all the configurations that were explored during the grid search. In the top-left graph, we can observe the validation accuracy achieved by each run. There is a clear decreasing trend over time, which suggests that the later configurations such as ResNet-101—tend to perform worse. This is expected since the sweep executes configurations in a fixed order. Finally, the last graph highlights the significant impact of the seed on the validation accuracy. The learning rate also plays a major role in the resulting loss, as does the number of frozen blocks. The red color indicates that as the value increases (e.g., more frozen blocks), performance tends to drop.

This should indicate overfitting and it does. But first, let's have a look at the run with the best performance.

| Model     | Frozen Layers | LR    | Val Acc (%) | Std Acc | Train Loss | Val Loss |
|-----------|---------------|-------|-------------|---------|------------|----------|
| ResNet-50 | 0             | 0.001 | 75.65       | 8.76    | 0.081      | 0.863    |
| ResNet-50 | 1             | 0.001 | 75.05       | 8.74    | 0.074      | 0.667    |
| ResNet-18 | 1             | 0.001 | 75.05       | 5.43    | 0.050      | 0.849    |
| ResNet-50 | 2             | 0.001 | 74.60       | 5.28    | 0.068      | 0.830    |
| ResNet-18 | 2             | 0.001 | 74.35       | 7.56    | 0.055      | 0.891    |

**Table 1:** Mean performance over 5 seeds for selected configurations. Constant hyperparameters are omitted for clarity.

The table below shows the averaged results over 5 different seeds for selected configurations. ResNet-50 with no frozen layers achieved the highest mean validation accuracy (75.65%), but it also presented the lowest training loss (0.081), which may indicate potential overfitting. This is especially visible when the training loss approaches zero, suggesting that the model may have memorized the 50 training examples per class. A run with ResNet-18 also demonstrated promising results across 5 seeds.

In traditional settings, such behavior would typically result in poor generalization: a model with near-zero training loss would often show high test loss and low test accuracy. However, in this case, the validation accuracy remains relatively high, which opens the door to an interpretations. The model has indeed learned strong and transferable features despite overfitting to the small training set.

To confirm whether these configurations are genuinely robust or just benefiting from lucky splits, the next step is to test them across a broader set of seeds and analyze the average and variance in their performance.

To do so, the following sweep configuration is used. It tries ResNet-18 and 50 on its most powerful setup and compare on 30 seeds each:

```

1 import pprint
2
3 # GRID SEARCH PRETRAINED
4
5 # Search method
6 sweep_config = {
7     'method': 'grid',
8     'name': "Verify_best_finetune_sweep"
9 }
10
11 # Objective
12 metric = {
13     'name': 'val_accuracy',
14     'goal': 'maximize'
15 }
16 sweep_config['metric'] = metric
17
18 # Parameters
19 parameters_dict = {
20     'seed': {
21         'values':
22             [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30]
23     },
24     'lr': {
25         'values': [0.001]
26     },
27     'batch_size': {
28         'values': [32]
29     },
30     'model_name': {
31         'values': ["resnet18", "resnet50"]
32     },
33     'freeze_layers': {
34         'values': [1]
35     },
36     'momentum': {

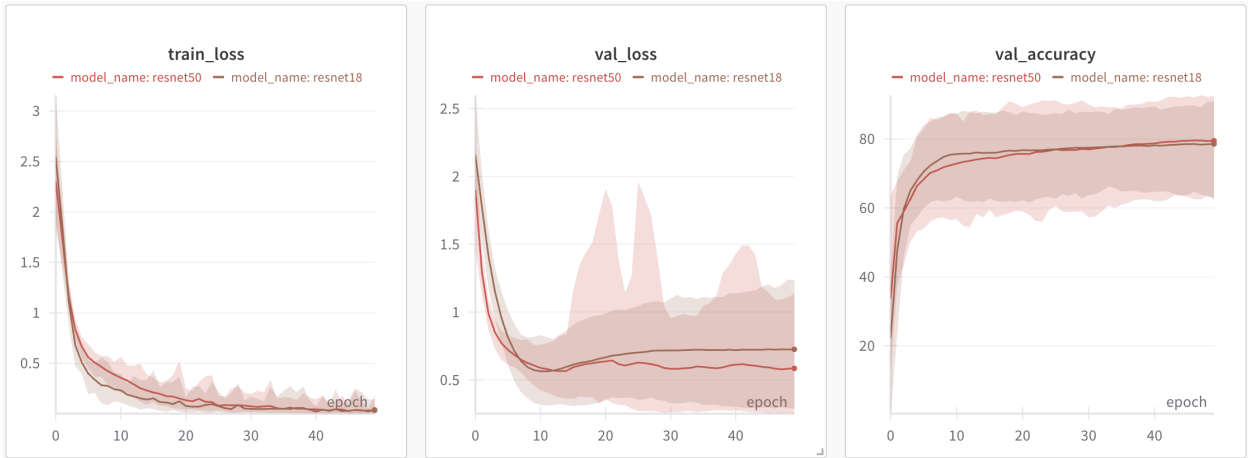
```

```

36         'values': [0.9]
37     },
38     'weight_decay': {
39         'values': [0.0005]
40     },
41     'epochs': {
42         'values': [50]
43     }
44 }
45 sweep_config['parameters'] = parameters_dict
46
47 parameters_dict.update({
48     'data_augmentation': {
49         'value': 1}
50 })
51
52 pprint.pprint(sweep_config)

```

The results for ResNet-18 and 50 on 30 different seeds are the following:



| Model     | Frozen Layers | Val Acc (%)      | Val Loss        | Train Loss      |
|-----------|---------------|------------------|-----------------|-----------------|
| ResNet-18 | 1             | $78.59 \pm 6.32$ | $0.72 \pm 0.24$ | $0.03 \pm 0.03$ |
| ResNet-50 | 1             | $79.51 \pm 7.63$ | $0.58 \pm 0.22$ | $0.04 \pm 0.04$ |

**Table 2:** Average performance over 30 seeds for the two final configurations.

The table above presents the mean performance of the two selected configurations over 30 different seeds. ResNet-50 with one frozen block slightly outperformed ResNet-18 in terms of validation accuracy (79.51% vs. 78.59%) while also achieving a lower validation loss. Although its standard deviation is slightly higher, its overall results suggest a better capacity to generalize. These two configurations remain the strongest candidates explored so far even if they are clearly overfitting.

*Additional Note:* Although better results have been obtained in the main code, the scores reported here are kept for consistency and comparison purposes due to high variance across seeds. The current best result, as reported in the submitted main code, is:

Acc over 30 instances: 81.22 +- 6.54

## MAML

As part of Challenge 2, I attempted to implement a meta-learning strategy using Model-Agnostic Meta-Learning (MAML). The goal was to test whether a model could learn to adapt rapidly to new 2-way classification tasks based on only a few examples per class. The code provided below defines a full MAML training loop built on top of a ResNet backbone (ResNet-18 and ResNet-50), using the CIFAR-10 dataset.

Despite what I suppose to be successful implementation, the performance of this approach remained limited. Validation accuracy tended to plateau between 65–70%, even after parameter sweeps involving different learning rates, backbone depths, and inner loop steps. It is possible that these results stem from suboptimal hyperparameter tuning or a misunderstanding of some of MAML’s specification—such as the appropriate gradient flow handling between inner and outer loops.

While this approach did not yield state-of-the-art results in my experiments, the full code and sweep configuration are available in the appendix for reference. This method is not part of the final submitted evaluation, but I chose to include it here to document the exploration effort.

The following section provides a breakdown of the MAML training code used in this project. The method was designed to perform few-shot learning on 2-way classification tasks using CIFAR-10.

### Device Setup

The code first detects the available device (MPS for Apple, CUDA for NVIDIA, or CPU fallback) to enable hardware-accelerated training.

```
1 # Device
2 if torch.backends.mps.is_available():
3     device = torch.device("mps")
4 elif torch.cuda.is_available():
5     device = torch.device("cuda")
6 else:
7     device = torch.device("cpu")
```

### Backbone Architecture

The `ResNetBackbone` class wraps a pre-trained ResNet (18, 34, or 50) from the PyTorch model zoo. The final classification layer is replaced with a custom linear layer for binary classification. The original `fc` head is removed and replaced by a trainable 2-class head (`nn.Linear(in_features, 2)`), making the model suitable for dynamic binary tasks.

```
1 # Build a ResNet-based model for 2-way classification
2 class ResNetBackbone(nn.Module):
3     def __init__(self, backbone_name='resnet18'):
4         super().__init__()
5         # load pretrained backbone
6         if backbone_name == 'resnet18':
7             self.model = models.resnet18(pretrained=True)
8         elif backbone_name == 'resnet34':
9             self.model = models.resnet34(pretrained=True)
10        elif backbone_name == 'resnet50':
11            self.model = models.resnet50(pretrained=True)
12        else:
13            raise ValueError(f"Unsupported backbone_{backbone_name}")
14        # remove original fc
```

```

15         in_feat = self.model.fc.in_features
16         self.model.fc = nn.Identity()
17         # new head for 2 classes
18         self.classifier = nn.Linear(in_feat, 2)
19
20     def forward(self, x):
21         feat = self.model(x)
22         return self.classifier(feat)

```

## Task Sampling

The function `sample_task` generates a random 2-way classification task from CIFAR-10. Two classes are selected at random, and for each class, it samples `n_support` images for training and `n_query` images for evaluation. These samples are packaged as PyTorch `DataLoader` objects, ensuring batch processing is efficient.

```

1  # Sample one MAML task (2-way few-shot)
2  def sample_task(dataset, classes, n_support, n_query, seed=None):
3      rng = np.random.RandomState(seed)
4      c1, c2 = rng.choice(classes, 2, replace=False)
5      idx = np.array(dataset.targets)
6      support_idx, query_idx = [], []
7      for c in (c1, c2):
8          c_idx = np.where(idx==c)[0]
9          perm = rng.permutation(c_idx)
10         support_idx.extend(perm[:n_support])
11         query_idx.extend(perm[n_support:n_support+n_query])
12     support_loader = DataLoader(Subset(dataset, support_idx), batch_size=len(
13         support_idx), shuffle=False)
14     query_loader = DataLoader(Subset(dataset, query_idx), batch_size=len(
15         query_idx), shuffle=False)
16     return support_loader, query_loader, [c1, c2]

```

## Meta-Training Loop

The function `train_maml` carries out the meta-training process. A sweepable `config` dictionary is used to control hyperparameters (with W&B). The outer loop iterates over `meta_epochs`, and in each epoch, the following steps are performed:

- **Task Sampling:** A batch of `tasks_per_epoch` few-shot tasks are generated.
- **Inner Loop Adaptation:** For each task, the base model is cloned (`deepcopy`) to simulate fast learning. This copy is trained on the support set using standard gradient descent for `inner_steps`.
- **Outer Loop Evaluation:** The adapted model is then evaluated on the query set. Loss and accuracy are aggregated across tasks.
- **Meta Update:** The average query loss is backpropagated through the meta-model, allowing it to learn parameters that are more amenable to quick adaptation in future tasks.

```

1  # MAML meta-training
2

```

```

3 def train_maml(config=None):
4     with wandb.init(config=config):
5         cfg = wandb.config
6         torch.manual_seed(cfg.seed)
7         np.random.seed(cfg.seed)
8         # data + transform
9         normalize = transforms.Normalize((0.4914,0.4822,0.4465)
10                                         ,(0.247,0.243,0.261))
11         transform = transforms.Compose([transforms.ToTensor(), normalize])
12         cifar = datasets.CIFAR10('.', train=True, transform=transform, download=
13             True)
14         classes = list(range(10))
15         # meta-model
16         meta_model = ResNetBackbone(cfg.backbone).to(device)
17         meta_opt = optim.Adam(meta_model.parameters(), lr=cfg.meta_lr)
18         # meta-training loop
19         for epoch in range(cfg.meta_epochs):
20             meta_opt.zero_grad()
21             total_loss = 0.0
22             total_acc = 0.0
23             for t in range(cfg.tasks_per_epoch):
24                 # sample task
25                 sup, qry, two = sample_task(
26                     cifar, classes, cfg.n_support, cfg.n_query,
27                     seed=cfg.seed + epoch*cfg.tasks_per_epoch + t)
28                 # clone for inner loop
29                 fast_model = deepcopy(meta_model)
30                 fast_opt = optim.SGD(fast_model.parameters(), lr=cfg.inner_lr)
31                 # inner adaptation
32                 fast_model.train()
33                 for _ in range(cfg.inner_steps):
34                     for x_s, y_s in sup:
35                         x_s, y_s = x_s.to(device), y_s.to(device)
36                         y_s = torch.tensor([0 if y==two[0] else 1 for y in y_s],
37                             device=device)
38                         logits = fast_model(x_s)
39                         loss = F.cross_entropy(logits, y_s)
40                         fast_opt.zero_grad(); loss.backward(); fast_opt.step()
41                 # query evaluation
42                 fast_model.eval()
43                 for x_q, y_q in qry:
44                     x_q, y_q = x_q.to(device), y_q.to(device)
45                     y_q = torch.tensor([0 if y==two[0] else 1 for y in y_q],
46                         device=device)
47                     logits_q = fast_model(x_q)
48                     lq = F.cross_entropy(logits_q, y_q)
49                     total_loss += lq
50                     total_acc += (logits_q.argmax(1)==y_q).float().mean()
51             # meta update
52             total_loss /= cfg.tasks_per_epoch
53             total_acc /= cfg.tasks_per_epoch
54             total_loss.backward()
55             meta_opt.step()

```

## W&B Integration

Each epoch's meta loss and meta accuracy are logged using Weights & Biases. The experiment is managed via a sweep configuration that systematically explores various values for seed, learning

rates (inner and outer), number of inner steps, and backbone type.

```
1 wandb.log({
2     'meta_epoch': epoch,
3     'meta_loss': total_loss.item(),
4     'meta_accuracy': total_acc.item()
5 })
```

## Sweep Configuration

The provided `sweep_config` defines a grid search over multiple values for:

- `meta_lr`, `inner_lr` (learning rates for outer and inner loops)
- `inner_steps` (how many SGD steps in the inner loop)
- `backbone` (either ResNet-18 or ResNet-50)
- `seed` (for reproducibility)

```
1 sweep_config = {
2     'name': 'MAML_sweep',
3     'method': 'grid',
4     'metric': {'name': 'meta_accuracy', 'goal': 'maximize'},
5     'parameters': {
6         'seed': {'values': [1, 2, 3]},
7         'backbone': {'values': ['resnet18', 'resnet50']},
8         'meta_lr': {'values': [1e-3, 1e-4]},
9         'inner_lr': {'values': [0.1, 0.01]},
10        'inner_steps': {'values': [5, 10]},
11        'meta_epochs': {'value': 20},
12        'tasks_per_epoch': {'value': 10},
13        'n_support': {'value': 25},
14        'n_query': {'value': 25}
15    }
16 }
```

Overall, this code provides a solid foundation for implementing MAML in a supervised few-shot classification setting. However, fine-tuning and more careful gradient tracking (e.g., higher-order gradients, learning rate annealing) may be required to achieve competitive results.

## Computation Time and Efficiency

In terms of computation time, my approach for Challenge 1 runs in roughly the same time as the provided baseline, while achieving a significantly better validation accuracy. This makes it a very efficient and effective improvement, especially considering the limited data and constraints of the task.

For Challenge 2, although the fine-tuned ResNet-50 model achieved slightly better accuracy than the scratch-trained network from Challenge 1, the improvement was modest. In contrast, the training time and memory usage were much higher due to the size of the backbone. This raises the question of whether the performance gain justifies the computational cost in a few-shot setting, and suggests that simpler models might be preferable in constrained environments.

## References

- [1] Weights & Biases. Experiment tracking with weights and biases. <https://wandb.ai/>, 2020. <https://wandb.ai/>.
- [2] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [5] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library, 2019.
- [6] Sachin Ravi and Hugo Larochelle. Optimization as a model for few-shot learning. In *International Conference on Learning Representations (ICLR)*, 2017.